ABC Company

Case Studies & Portfolio

ɔcument ID:	ABC-DOC-003
rsion:	v1.0
ıte:	June 2026
assification:	Business Confidential

ABC Company — Colombo | London | Melbourne

Table of Contents

Portfolio Summary

The following table provides a high-level overview of the eight client engagements documented in this portfolio. Each case study is presented in full detail in the sections that follow, covering the client challenge, ABC Company's solution approach, key quantitative outcomes, and a client testimonial.

Project	Industry	Duration	Team Size	Key Metric	Technologies
NovaPay	Financial Technology	9 months	7	€2M → €8M daily volume	Node.js, NestJS, React, PostgreSQL, Redis, AWS, Kubernetes, Stripe
MediConnect	Healthcare	7 months	7	3,500 patients in 60 days	Python, FastAPI, React, React Native, PostgreSQL, AWS, HL7/FHIR
TradeNest	E-Commerce	11 months	9	120% GMV growth in 6 months	Next.js, Node.js, PostgreSQL, Elasticsearch, Redis, AWS, Stripe Connect
InsightFlow	SaaS / Data Analytics	8 months	7	$2.4M ARR in 12 months	React, D3.js, Python, FastAPI, Kafka, ClickHouse, PostgreSQL, AWS
SwiftRoute	Logistics & Supply Chain	10 months	7	€400K annual fuel savings	React, React Native, Node.js, NestJS, PostgreSQL, PostGIS, AWS IoT, MQTT
LearnSpark	Education Technology	12 months	9	92% course completion rate	React, Python, Django, PostgreSQL, TensorFlow, WebRTC, AWS, Redis
DwellDesk	Property Technology	6 months	6	97% on-time rent collection	React, React Native, Node.js, PostgreSQL, Stripe, Twilio, AWS
GlobalMed Supplies	Enterprise / Healthcare Supply Chain	5 months	5	3.5-month ROI, 83% headcount reduction in data ops	Python, FastAPI, React, Salesforce API, SAP, PostgreSQL, Redis, AWS

Case Study 1: NovaPay — FinTech Payment Platform

Client	NovaPay (EU-based fintech startup)
Industry	Financial Technology
Duration	9 months
Team Size	7 (2 Backend Engineers, 2 Frontend Engineers, 1 DevOps, 1 QA, 1 Tech Lead)
Technologies	Node.js, NestJS, React, PostgreSQL, Redis, AWS, Docker, Kubernetes, Stripe API, PCI-DSS compliance tooling

The Challenge

NovaPay was a seed-stage fintech startup with an ambitious vision: to disrupt business-to-business payment flows across the European Union by offering faster settlement, richer transaction data, and competitive foreign exchange rates compared to incumbent banking rails. Their initial prototype, assembled by a solo developer over six months, had found genuine market traction — processing approximately €2M per day across a small cohort of early merchant partners. However, the cracks in the foundation were becoming impossible to ignore. Transaction failure rates hovered near 8%, a number that was commercially unacceptable at scale. The monolithic Node.js application had no separation between payment processing, merchant management, and reporting concerns, meaning that a bug in the reconciliation module could take down the entire payment flow.

Beyond reliability, NovaPay faced an existential regulatory challenge: PCI-DSS Level 1 compliance. Processing volumes above a certain threshold mandated full Level 1 certification, and NovaPay's investor pipeline for their Series A was conditional on demonstrating that compliance before term sheets could progress. The existing codebase stored raw card data in an unencrypted PostgreSQL column — a finding that would have been an immediate critical failure under any formal PCI-DSS audit. The founders came to ABC Company with three clear mandates: rebuild the core payment processing engine with the reliability and security architecture their growth demanded, achieve PCI-DSS Level 1 certification within six months, and ship a merchant-facing dashboard that would enable self-service transaction monitoring, reconciliation, and dispute management.

Our Solution

ABC Company's engagement began with a two-week discovery phase during which our tech lead and two senior backend engineers performed a full audit of the existing codebase, mapped every payment flow, and catalogued all third-party integrations. The architectural recommendation was a clean-slate microservices redesign built on NestJS — a TypeScript-first framework whose module system maps cleanly onto service boundaries. We decomposed the monolith into five core services: the Payment Processor (responsible for Stripe API orchestration, idempotency key management, and charge lifecycle), the Merchant Service (profile management, API key issuance, and rate configuration), the Reconciliation Service (end-of-day settlement calculations, payout scheduling, and ledger management), the Notification Service (webhook dispatch to merchant endpoints), and the Audit Service (immutable transaction event log consumed by all other services via a message queue). PostgreSQL was retained as the primary transactional data store, with Redis providing distributed session management, idempotency key caching, and rate-limiting state across the fleet of horizontally scaled service pods.

The PCI-DSS compliance programme ran in parallel with the architectural rebuild. Our team implemented tokenization at the point of card data capture, ensuring that raw Primary Account Numbers never touched NovaPay's infrastructure — card data was vaulted directly with Stripe's PCI-certified tokenization service, and only opaque tokens were stored

in NovaPay's database. All data at rest was encrypted using AES-256 with AWS KMS-managed keys. All service-to-service communication was enforced over mutual TLS. We deployed HashiCorp Vault for secrets management, eliminating hardcoded credentials from the codebase entirely. Comprehensive audit logging captured every privileged action in an append-only log stream. Access controls were redesigned around least-privilege principles, with network segmentation enforced at the Kubernetes namespace level. ABC Company coordinated the quarterly external penetration test and the on-site QSA assessment, preparing the evidence portfolio that ultimately secured NovaPay's PCI-DSS Level 1 Report on Compliance.

The migration from monolith to microservices was executed over six weeks using a strangler-fig pattern designed to achieve zero downtime for existing merchant partners. A thin API gateway layer was introduced in front of both the legacy monolith and the new services, routing traffic to the new services on a per-endpoint basis as each service passed a two-week parallel-run validation against production traffic shadows. At no point were merchant transactions interrupted. The cutover was completed endpoint by endpoint, with the monolith decommissioned once all routes had been migrated and validated. The React-based merchant dashboard delivered real-time transaction feeds via server-sent events, a reconciliation module with CSV/PDF export, fraud detection alert banners surfacing anomalies from the Audit Service, and a dispute management workflow enabling merchants to submit and track chargebacks without contacting support.

Key Results

DAILY TRANSACTION VOLUME

€8M

UP FROM €2M (4× GROWTH)

TRANSACTION FAILURE RATE

0.3%

DOWN FROM 8% (96% REDUCTION)

COMPLIANCE

Level 1

PCI-DSS LEVEL 1 CERTIFIED

PLATFORM UPTIME

99.97%

MEASURED OVER FIRST 6 MONTHS

Client Testimonial

> *ABC Company didn't just rebuild our platform — they transformed our business. Their deep understanding of payment systems and regulatory requirements gave us the confidence to pursue our Series A, which we closed at €12M three months after launch.*
>
> **— CTO, NovaPay**

Case Study 2: MediConnect — Healthcare Patient Portal

Client	MediConnect (healthcare network, 12 clinics)
Industry	Healthcare
Duration	7 months
Team Size	7 (1 Tech Lead, 2 Backend, 2 Frontend, 1 Mobile Dev, 1 QA)
Technologies	Python, FastAPI, React, React Native, PostgreSQL, AWS (HIPAA-eligible services), HL7/FHIR integration

The Challenge

MediConnect operated a network of twelve clinics across two metropolitan areas, collectively managing over 50,000 active patient records. Despite the scale of the operation, the clinics' technology infrastructure was a patchwork of legacy systems accumulated through years of organic growth and clinic acquisitions. Three separate Electronic Health Record (EHR) systems were in use across the network, none of which communicated with the others. Patient scheduling was handled by a combination of telephone calls and a basic web form that pushed appointments into a shared spreadsheet reviewed manually by clinic administrators each morning. Medical records were stored in a mix of paper files and proprietary EHR silos, making it virtually impossible for a consulting physician at one clinic to access records from another without a phone call and a faxed document.

The clinical and operational consequences were significant. No-show rates averaged 24%, representing thousands of wasted appointment slots annually. Patients frequently arrived at appointments without their records having been forwarded from their previous clinic visit, forcing physicians to repeat diagnostic workups. Prescription refill requests required patients to physically visit the clinic or leave a voicemail. MediConnect's CEO had articulated a three-year vision for the network: a unified digital platform that would allow patients to self-serve, give clinicians a complete longitudinal record at the point of care, and allow the network's administrative staff to manage the entire portfolio from a single web dashboard. Every component of this vision had to be built on a foundation of full HIPAA compliance, with the added complexity that AWS Business Associate Agreements needed to be established for every service touching Protected Health Information.

Our Solution

ABC Company designed a HIPAA-compliant architecture exclusively using AWS services covered under Amazon's BAA programme. All PHI was stored in encrypted RDS PostgreSQL instances with automated backups to S3 with server-side encryption. Application-level audit trails captured every read and write to patient records, stored in a separate append-only audit database with access restricted to the compliance role. Network architecture used private subnets for all data stores, with no direct public ingress. Role-based access control was implemented with four distinct roles — Patient, Nurse, Physician, and Administrator — each with strictly scoped permissions defined at both the application and database levels. The Python/FastAPI backend was chosen for its performance, Pydantic-based data validation (which enforced schema contracts on all PHI fields), and the team's deep familiarity with the stack.

The EHR integration layer was the most technically complex component of the project. The three existing EHR systems in use across the MediConnect network — Athenahealth, eClinicalWorks, and a custom legacy system — each exposed different API surfaces. ABC Company implemented a unified HL7 FHIR R4 integration layer that translated each EHR's native API into a consistent internal data model. Patient demographics, clinical notes, lab results, medication lists, and immunisation records could now be accessed through a single API endpoint, regardless

of which EHR held the source record. A bidirectional sync mechanism propagated updates made through the new patient portal back to the source EHR, ensuring that the legacy systems remained the system of record for clinical purposes while the new platform served as the patient-facing interface.

The patient portal delivered online appointment scheduling with real-time slot availability pulled from the clinic calendar systems, secure messaging with care team members via an encrypted inbox, access to medical records and lab results, and a one-click prescription refill request flow routed to the responsible physician for approval. A React Native mobile application mirrored the portal's core functionality on iOS and Android, with biometric authentication and push notification support for appointment reminders. Automated SMS and push reminders, sent 48 hours and 2 hours before each appointment, were the primary driver of the dramatic reduction in no-show rates observed after launch. The clinic staff dashboard provided occupancy analytics, patient communication history, maintenance of provider schedules, and a referral management workflow for inter-clinic patient transfers.

Key Results

PATIENTS ONBOARDED

3,500

IN FIRST 60 DAYS POST-LAUNCH

NO-SHOW RATE REDUCTION

67%

VIA AUTOMATED APPOINTMENT REMINDERS

PATIENT SATISFACTION

4.8/5

AVERAGE RATING ACROSS 12 CLINICS

COMPLIANCE

HIPAA

FULL HIPAA COMPLIANCE CERTIFIED

Client Testimonial

> *The team at ABC Company navigated the complexity of healthcare regulations while delivering a product our patients love. The integration with our existing EHR systems was seamless — something three previous vendors had told us was impossible.*
>
> **— Director of Digital Health, MediConnect**

Case Study 3: TradeNest — E-Commerce Marketplace

Client	TradeNest (B2B wholesale marketplace, 200+ sellers)
Industry	E-Commerce
Duration	11 months
Team Size	9 (1 Tech Lead, 3 Backend, 2 Frontend, 1 DevOps, 1 QA, 1 Designer)
Technologies	Next.js, Node.js, PostgreSQL, Elasticsearch, Redis, AWS, Stripe Connect, Algolia

The Challenge

TradeNest had built a functioning B2B wholesale marketplace connecting over two hundred verified suppliers with several thousand registered buyers across the manufacturing, retail, and hospitality sectors. The business model was sound — take rates on marketplace transactions generated meaningful revenue — but the platform was haemorrhaging potential transactions due to a combination of severe performance problems and a buyer experience that had not been meaningfully updated since the platform's launch three years earlier. Page load times averaged eight seconds on the product catalogue pages that drove the majority of buyer sessions, a figure that far exceeded the three-second threshold beyond which B2B buyers reliably abandon browsing sessions according to internal conversion analytics.

The cart abandonment rate of 68% told a parallel story. Buyers who had invested time finding products and adding them to a cart were failing to convert because the checkout flow required re-entering shipping details on every order, because the platform lacked saved payment methods, and because the multi-step checkout sequence was poorly optimised for the bulk-order dynamics of B2B purchasing, where buyers frequently need to adjust quantities, verify pricing tiers, and submit orders for internal approval before payment. The underlying architecture was a PHP monolith with a MySQL database — a stack that had served the early platform well but whose tightly coupled nature meant that even small feature changes required testing the entire application, stretching release cycles to four to six weeks. TradeNest's CEO engaged ABC Company with a clear brief: make the platform fast, modern, and capable of scaling to support ten times the current transaction volume within eighteen months.

Our Solution

ABC Company's frontend team rebuilt the buyer-facing interface from scratch using Next.js, leveraging Server-Side Rendering for product listing pages to ensure fast initial page loads for search-engine-crawled content, and Incremental Static Regeneration for the product detail pages that accounted for the majority of catalogue traffic. A critical optimisation was the implementation of a CDN-cached ISR layer that served the most frequently accessed product pages from edge nodes geographically distributed to serve TradeNest's European and Asia-Pacific buyer base. The result was a reduction in average page load time from eight seconds to 1.8 seconds, measured across the P95 percentile of real-user monitoring data collected during the post-launch period.

The search overhaul addressed one of the marketplace's most significant friction points: discoverability. The legacy keyword-matching search was replaced with an Elasticsearch cluster backed by an Algolia search-as-you-type layer for the autocomplete experience. Faceted search was implemented across product category, supplier, minimum order quantity, unit price bands, lead time, and certification attributes — allowing B2B buyers to drill down to exactly the SKUs relevant to a specific sourcing need in a way that was simply not possible before. Search conversion rate — the percentage of searches that resulted in an add-to-cart event — tripled in the three months following launch.

The backend migration decomposed the PHP monolith into four Node.js microservices: Order Management, Inventory & Catalogue, Payments (using Stripe Connect for marketplace-native split payments to suppliers), and Notifications. The checkout flow was redesigned around B2B buyer workflows: one-click reorder for repeat purchases, saved cart persistence across sessions, a configurable approval workflow for buyers whose organisations required internal sign-off before order submission, and a bulk order upload via CSV for buyers placing large assortments. The seller dashboard was rebuilt with a React SPA offering real-time order notifications, inventory management with low-stock alerts, fulfilment workflow management, and a seller analytics module showing sales trends, top-performing SKUs, and buyer retention metrics.

Key Results

PAGE LOAD TIME

1.8s

DOWN FROM 8S AVERAGE (78% FASTER)

CART ABANDONMENT

−54%

REDUCTION IN ABANDONMENT RATE

SEARCH CONVERSION

3×

INCREASE IN SEARCH-TO-CART RATE

GMV GROWTH

120%

MARKETPLACE GMV OVER 6 MONTHS

Client Testimonial

> *The performance improvements alone paid for the project within three months. Our sellers are selling more, our buyers are buying more, and our platform finally feels like it belongs in the same decade as our competitors.*
>
> **— CEO, TradeNest**

Case Study 4: InsightFlow — SaaS Analytics Dashboard

Client	InsightFlow (SaaS analytics startup)
Industry	SaaS / Data Analytics
Duration	8 months
Team Size	7 (1 Tech Lead, 2 Backend, 2 Frontend, 1 Data Engineer, 1 QA)
Technologies	React, D3.js, Python, FastAPI, PostgreSQL, Apache Kafka, ClickHouse, Redis, AWS

The Challenge

InsightFlow's founders had identified a genuine gap in the mid-market business intelligence landscape: enterprise-grade analytics platforms like Tableau and Looker were too expensive and too complex for the growing cohort of mid-sized SaaS companies generating meaningful operational data but lacking the data engineering resources to operationalise it. InsightFlow's vision was a self-serve analytics platform that could ingest data from the tools these companies already used — Salesforce, HubSpot, Google Analytics, Stripe, and custom product databases — and surface actionable insights through beautiful, interactive dashboards without requiring a data scientist to configure them.

The founders had validated the concept with a prototype that attracted a waitlist of over three hundred companies. However, the prototype's technical constraints were severe. It could handle a maximum of one hundred concurrent users before response times degraded to an unusable level. Data pipelines ran as nightly batch jobs, resulting in a minimum data latency of thirty minutes and, in practice, often closer to several hours when jobs failed and were retried. The database underpinning the query engine was a standard PostgreSQL instance that was simply not designed for the analytical query patterns InsightFlow needed — aggregations across tables with hundreds of millions of rows were taking minutes to return. InsightFlow needed ABC Company to design and build the production-grade data architecture that would turn their validated concept into a commercially deployable product.

Our Solution

The centrepiece of ABC Company's solution was a purpose-built event streaming and analytical query architecture. Apache Kafka was deployed as the event ingestion backbone, with dedicated Kafka Connect connectors built for each of the initial data sources — Salesforce, HubSpot, Google Analytics, Stripe, and a generic REST API connector for custom product databases. Each connector published normalised event streams to Kafka topics, which were consumed by transformation workers that applied client-specific field mappings and enrichment logic before writing to the analytical store. ClickHouse was selected as the OLAP query engine based on benchmarks showing sub-second response on aggregation queries across datasets exceeding one hundred million rows — a performance profile that PostgreSQL simply could not match for the analytical workloads InsightFlow required. Redis provided the caching layer for frequently requested dashboard snapshots, further reducing query load during peak usage.

The dashboard engine was built as a React SPA with WebSocket connections providing real-time data push for live-updating charts. The visualisation layer used D3.js for custom chart types alongside a library of pre-built chart components covering time-series, funnel, cohort, and geographic visualisations. A custom chart builder allowed users to compose multi-metric dashboards through a drag-and-drop interface without writing any configuration code. The Python/FastAPI backend exposed a unified query API that translated dashboard configuration objects into optimised ClickHouse SQL, handling query planning, caching, and result serialisation. The multi-tenant architecture enforced

strict data isolation at the database level through ClickHouse's tenant-partitioned table design, ensuring that query plans for one tenant could never inadvertently scan another tenant's data.

The data connector framework was designed as a plug-in architecture from the outset, with a well-defined connector interface that allowed new data sources to be added without modifying the core platform. Each connector was independently deployable as a container, registered itself with the connector registry at startup, and exposed a standardised configuration schema that the InsightFlow onboarding UI rendered dynamically — enabling customers to connect a new data source through a guided wizard in under five minutes. InsightFlow's enterprise customers also benefited from per-tenant custom branding — logo, colour scheme, and domain — making it possible for InsightFlow to offer a white-label tier to agency and reseller partners, a revenue stream that contributed materially to the $2.4M ARR figure achieved within twelve months of the production launch.

Key Results

DAILY ACTIVE USERS	ARR ACHIEVED	QUERY RESPONSE
10,200	**$2.4M**	**<1s**
AT 12 MONTHS POST-LAUNCH	WITHIN 12 MONTHS OF LAUNCH	ON 100M+ ROW DATASETS

UPTIME SLA

99.95%

MAINTAINED ACROSS ALL TENANTS

Client Testimonial

> *ABC Company turned our napkin sketch into a product that enterprises actually pay for. Their data engineering expertise was critical — we went from batch processing to real-time analytics in just three months, which was the difference between losing our first enterprise customer and landing a six-figure annual contract.*
>
> **— Founder & CEO, InsightFlow**

Case Study 5: SwiftRoute — Logistics Fleet Management

Client	SwiftRoute (logistics company, 500+ vehicles, 3 countries)
Industry	Logistics & Supply Chain
Duration	10 months
Team Size	7 (1 Tech Lead, 2 Backend, 1 Frontend, 1 Mobile Dev, 1 DevOps, 1 QA)
Technologies	React, React Native, Node.js, NestJS, PostgreSQL, PostGIS, Redis, Google Maps API, MQTT, AWS IoT

The Challenge

SwiftRoute operated a fleet of more than five hundred delivery vehicles across logistics hubs in three countries, managing last-mile and middle-mile delivery contracts for manufacturing, retail, and pharmaceutical clients. Despite the scale of the operation, the company's technology infrastructure was striking in its absence of digital capability. Dispatch was handled by a team of twelve coordinators who assigned jobs by reviewing paper job sheets spread across a central dispatch table, allocating routes to drivers through a combination of personal knowledge of driver locations and phone calls to confirm availability. Drivers received their manifests as printed sheets handed over at the depot each morning, and any changes during the day — additional stops, cancelled deliveries, urgent re-routes — were communicated by mobile phone.

The business consequences of this manual operation were measurable and severe. On-time delivery rates averaged 71% against client SLA targets of 95%, exposing the company to significant penalty clauses in its logistics contracts. Fuel consumption was running approximately 23% above the fleet's theoretical optimum because routes were planned on driver familiarity rather than dynamic optimisation. Proof of delivery was paper-based, meaning that client billing disputes required retrieving physical documents from the depot archive — a process that could take days and frequently resulted in SwiftRoute absorbing disputed charges rather than pursuing them. SwiftRoute's COO commissioned ABC Company to design and build a full fleet management and dispatch platform that would give the company operational visibility and intelligence across its entire vehicle network in real time.

Our Solution

The technical foundation of the SwiftRoute platform was a real-time GPS tracking layer built on AWS IoT Core and an MQTT message broker. Each vehicle was fitted with an OBD-II telematics device that published location, speed, engine status, and fuel consumption telemetry at thirty-second intervals. The MQTT broker ingested these streams and wrote them to a PostGIS-enabled PostgreSQL database, enabling geospatial queries such as 'find all vehicles within 5km of this delivery point' and 'calculate the actual route travelled by vehicle X between 09:00 and 14:00'. A Node.js/NestJS backend exposed this geospatial data through a REST and WebSocket API consumed by both the dispatch dashboard and the driver mobile application.

The route optimisation engine processed each day's delivery manifest the evening before dispatch, applying a constrained vehicle routing problem solver that considered delivery time windows, vehicle payload capacity, historical traffic patterns by time of day and day of week (sourced from the Google Maps Roads API), and driver working hours regulations. For intra-day disruptions — a new urgent delivery, a driver reporting vehicle downtime, a customer requesting a delivery window change — the engine could re-optimise affected routes in under thirty seconds, with the updated manifests pushed instantly to the relevant driver's mobile app. ABC Company's data analysis of the first three months of live operation confirmed a 23% improvement in route efficiency compared to the pre-platform baseline.

The driver mobile application, built with React Native for simultaneous iOS and Android deployment, provided turn-by-turn navigation integrated with the day's optimised manifest, digital proof-of-delivery capture (signature, photo, GPS timestamp, and recipient name), in-app messaging with the dispatch team, and exception reporting for failed delivery attempts. The dispatch dashboard presented a real-time fleet map with vehicle positions, delivery status, and SLA adherence indicators, plus automated job assignment recommendations generated by the optimisation engine. An analytics module provided operations management with daily, weekly, and monthly reports on fuel consumption by vehicle, driver performance scores, delivery SLA adherence by client contract, and exception root-cause analysis.

Key Results

ROUTE EFFICIENCY

+23%

IMPROVEMENT VS. PRE-PLATFORM BASELINE

ON-TIME DELIVERY

94%

UP FROM 71% (32% IMPROVEMENT)

FUEL COST REDUCTION

18%

€400K ANNUAL SAVINGS

DISPATCH AUTOMATION

85%

REDUCTION IN MANUAL DISPATCH TIME

Client Testimonial

> *SwiftRoute went from manual chaos to a fully digital operation in under a year. The route optimisation alone saves us €400,000 annually in fuel costs, which is a figure our CFO still cannot quite believe. The on-time delivery improvement meant we avoided over €200,000 in contract penalty clauses in the first year alone.*
>
> **— VP Operations, SwiftRoute**

Case Study 6: LearnSpark — EdTech Learning Platform

Client	LearnSpark (education technology company)
Industry	Education Technology
Duration	12 months
Team Size	9 (1 Tech Lead, 3 Backend, 2 Frontend, 1 ML Engineer, 1 Designer, 1 QA)
Technologies	React, Python, Django, PostgreSQL, TensorFlow, AWS, WebRTC, Redis, Celery

The Challenge

LearnSpark's founding team — three former university lecturers and an educational psychologist — had a research-backed hypothesis: that the failure of online learning platforms to achieve meaningful learning outcomes was primarily a pacing and personalisation problem. Existing platforms presented all students with the same content in the same sequence at the same pace, ignoring the vast variation in prior knowledge, learning speed, and preferred modality that characterised any real student population. LearnSpark wanted to build a platform that genuinely adapted — one where the system's model of each student's knowledge state continuously updated as the student engaged with content, and where the next piece of content served was always the one most likely to advance that specific student's learning.

Beyond the adaptive learning engine, the platform required a full suite of online education infrastructure: live virtual classroom capabilities to replace the synchronous teaching that institutional customers were unwilling to entirely abandon, a content authoring system enabling their institutional clients to build courses without engineering support, a collaborative peer learning environment, and an automated assessment engine that could provide immediate formative feedback at the scale of fifty thousand concurrent users during end-of-term exam periods. ABC Company was engaged to architect and build the entire platform from the ground up, with a twelve-month timeline to the first institutional customer launch.

Our Solution

The adaptive learning engine was the most intellectually challenging component of the project. ABC Company's ML engineer implemented a knowledge tracing model based on a Bayesian knowledge graph, where each learner's mastery probability for each knowledge concept was continuously updated as they answered assessment questions and engaged with content modules. The model was trained on anonymised learning interaction data from LearnSpark's pilot programme and validated against a held-out test set before production deployment. The recommendation layer selected the next content module for each student by maximising the expected learning gain given the current knowledge state estimate, factoring in content prerequisites, the student's estimated cognitive load, and time constraints. The result was a personalisation engine that could serve materially different learning sequences to different students enrolling in the same course, without any manual configuration by the course instructor.

The virtual classroom was built on a WebRTC infrastructure deployed on AWS, supporting up to two hundred concurrent participants per session with adaptive bitrate video, an interactive shared whiteboard built with Fabric.js, configurable breakout rooms for small group work, instructor-controlled participant muting and screen-share permissions, and a polling and Q&A panel for real-time formative assessment during live sessions. Session recordings were automatically transcribed and indexed, enabling students to search live session content by keyword — a feature that proved particularly valued by students with hearing impairments and non-native speakers reviewing sessions after the fact.

The content authoring system provided course instructors with a drag-and-drop module builder supporting video, audio, text, interactive quizzes, coding exercises with automatic grading, and peer review assignments. The assessment engine supported multiple question types, randomised question banks to reduce academic dishonesty, time-limited assessments, and an automated plagiarism detection integration. The auto-scaling architecture, deployed on AWS ECS with pre-warming scheduled for known high-traffic periods such as exam weeks and course enrolment deadlines, successfully handled the peak load of 52,000 concurrent users recorded during the platform's first major end-of-term assessment period — with no degradation in response time compared to baseline performance.

Key Results

ACTIVE STUDENTS

52,000

ACROSS 15 INSTITUTIONS

PASS RATE IMPROVEMENT

+31%

WITH AI-PERSONALISED LEARNING PATHS

STUDENT SATISFACTION

4.7/5

AVERAGE ACROSS ALL INSTITUTIONS

COURSE COMPLETION RATE

92%

VS. 45% INDUSTRY AVERAGE

Client Testimonial

> *The AI-powered learning paths are a game changer. Students who were struggling are now passing at rates we never thought possible. ABC Company's machine learning expertise made this a reality, and the virtual classroom platform has completely replaced our previous video conferencing cobble-together. This is genuinely the platform we imagined when we founded the company.*
>
> **— Chief Learning Officer, LearnSpark**

Case Study 7: DwellDesk — PropTech Property Management

Client	DwellDesk (property management company, 2,000+ units)
Industry	Property Technology
Duration	6 months
Team Size	6 (1 Tech Lead, 2 Backend, 1 Frontend, 1 Mobile Dev, 1 QA)
Technologies	React, React Native, Node.js, PostgreSQL, Stripe, Twilio, AWS

The Challenge

DwellDesk Property Management oversaw a portfolio of more than two thousand residential rental units across three cities, operating under a business model in which property owners contracted DwellDesk to manage all aspects of tenancy — from marketing vacant units and screening applicants, through lease management and rent collection, to maintenance coordination and eventual vacancy turnover. The business had grown rapidly through acquisition of smaller property management firms, and the technology infrastructure had never kept pace. Rent collection was managed through a combination of mailed invoices, standing bank transfer instructions issued to tenants at lease signing, and a manual reconciliation process in which a team of three accounts staff matched inbound bank transfers to unit ledgers each morning using a shared Excel spreadsheet.

The on-time rent collection rate of 78% was well below the industry benchmark, and the accounts team was spending an estimated sixty hours per week on reconciliation alone. Maintenance requests arrived via a single property manager's mobile phone, were written into a personal notebook, and were followed up with vendor phone calls — a system that had no accountability, no SLA tracking, and no visibility for tenants who had no way to know whether their request had been received, assigned, or scheduled. Tenant communications were fragmented across email, SMS, phone calls, and occasional in-person visits, with no central history. DwellDesk's managing director engaged ABC Company to build a purpose-designed property management platform that would automate the manual workflows consuming her team's time and deliver a digital-first tenant experience.

Our Solution

The tenant-facing portal was built as a React web application with a companion React Native mobile app, providing tenants with a unified interface for all interactions with DwellDesk. The rent payment module supported ACH bank transfers and credit/debit card payments processed through Stripe, with automated recurring payment setup that allowed tenants to enrol in autopay at lease signing. A dynamic rent ledger showed tenants their current balance, payment history, and any outstanding charges in real time. Automated payment reminder notifications were sent via push notification and SMS through Twilio's messaging API at configurable intervals before each rent due date — 7 days, 3 days, and 1 day in advance by default, with an immediate notification for any failed payment.

The maintenance workflow management system replaced the property manager's notebook with a structured digital process. Tenants submitted maintenance requests through the portal with a description, category tag, urgency indicator, and optional photo attachments stored in S3. The request entered a triage queue visible to all property managers, with priority scoring based on urgency and category — a water leak triggering a different default SLA than a request to replace a light bulb. Property managers assigned approved requests to vendors from a pre-qualified vendor database, with vendor assignments generating automated work order notifications via SMS and email. Tenants received status update notifications at each stage of the workflow — assigned, scheduled, completed — with

a satisfaction rating request sent twenty-four hours after closure. The average maintenance response time fell from 9.2 days to 3.7 days in the first quarter of live operation.

The property manager dashboard provided a portfolio-wide occupancy view with unit-level drill-down, a financial reporting module generating monthly owner statements, a lease management tracker with automated renewal reminders, and a communication hub consolidating all tenant messaging into a single inbox with conversation history. An automated vacancy marketing workflow triggered a listing generation checklist when a lease was marked as expiring, prompting the creation of an advertising listing populated from the unit's profile. Financial reporting produced tax-ready income and expense summaries for property owners, replacing a quarterly manual process that had previously required a part-time bookkeeper.

Key Results

ON-TIME RENT COLLECTION

97%

UP FROM 78% (24% IMPROVEMENT)

MAINTENANCE RESPONSE TIME

−60%

9.2 DAYS → 3.7 DAYS AVERAGE

PROPERTY MANAGER WORKLOAD

−45%

THROUGH TASK AUTOMATION

ANNUAL ADMIN SAVINGS

$180K

IN ADMINISTRATIVE COST REDUCTION

Client Testimonial

> *DwellDesk was a nightmare of spreadsheets and phone calls. ABC Company gave us a modern platform that our tenants love and our property managers cannot live without. The rent collection improvement from 78% to 97% on time was visible within the first month, and the maintenance tracking alone has saved us more than one angry tenant from leaving.*
>
> **— Managing Director, DwellDesk**

Case Study 8: GlobalMed Supplies — Enterprise CRM Integration

Client	GlobalMed Supplies (medical supply distributor, $50M ARR)
Industry	Enterprise / Healthcare Supply Chain
Duration	5 months
Team Size	5 (1 Tech Lead, 2 Backend, 1 Frontend, 1 QA)
Technologies	Python, FastAPI, React, Salesforce API, SAP integration, PostgreSQL, Redis, AWS

The Challenge

GlobalMed Supplies was a well-established medical equipment and consumables distributor generating approximately $50M in annual revenue, serving hospitals, outpatient clinics, and medical research institutions across the Asia-Pacific region. Despite its commercial success, the company's internal technology landscape was a source of growing operational pain. Salesforce served as the CRM for the company's twenty-strong sales team, managing account records, opportunities, and customer communications. SAP was the ERP backbone, handling purchase orders, inventory, invoicing, and financial reporting. A separate custom-built web application managed pricing agreements and discount schedules negotiated with individual customers — agreements complex enough that a standard price list was insufficient, with tier discounts, volume rebates, and contract-specific line-item pricing across a catalogue of more than twelve thousand SKUs.

The fundamental problem was that these three systems did not talk to each other. A sales representative closing an opportunity in Salesforce had to manually re-enter the order details into SAP to create a sales order, cross-referencing the pricing application to apply the correct contract rates. A customer service agent looking at a Salesforce contact record had no visibility into that customer's order history, outstanding invoices, or inventory availability without switching to SAP and performing a separate search. Six full-time data entry staff were employed whose primary function was to reconcile data between the three systems — identifying discrepancies, correcting keying errors, and ensuring that Salesforce opportunities matched the SAP orders they had ostensibly generated. Order error rates averaged 12%, with incorrect pricing or wrong product codes being the most common failure modes. Generating a complete customer 360 view — order history, current opportunities, pricing agreements, service tickets, and outstanding invoices — required a manual compilation exercise taking up to forty-eight hours.

Our Solution

ABC Company designed and built an integration hub implemented as a Python/FastAPI middleware service deployed on AWS. The hub maintained authoritative bi-directional connectors to both Salesforce (via the Salesforce REST and Bulk APIs) and SAP (via SAP's OData REST API layer). A PostgreSQL operational database within the hub served as the integration state store, tracking the synchronisation status of every entity across the three source systems and providing a reconciliation layer that could detect and resolve conflicts according to configurable business rules — for example, specifying that in the event of a price discrepancy between Salesforce and the pricing application, the pricing application always takes precedence.

Real-time data synchronisation was achieved through a combination of Salesforce change data capture events (delivered via Salesforce's streaming API to a Kafka topic consumed by the hub) and SAP BAPI-triggered outbound calls to the hub's webhook endpoint. The median end-to-end sync latency measured in production was under four

seconds for individual record changes, with bulk synchronisation jobs handling overnight batch reconciliation for high-volume entity types. The hub's conflict resolution engine logged every conflict and resolution decision to an audit trail, providing the data stewardship team with full visibility into cases where automated rules had been applied and flagging edge cases requiring human review.

The unified customer dashboard was built as a React single-page application, providing sales and service staff with a single-pane-of-glass view of each customer account: Salesforce contact and opportunity data, full SAP order history with line-item detail and delivery status, active pricing agreements from the pricing application, outstanding invoices with payment status, and open service tickets. An automated order pipeline connected Salesforce opportunity closure directly to SAP order creation, with an approval workflow for orders above configurable value thresholds and automated application of the customer's negotiated pricing from the pricing application. Data quality tooling applied automated deduplication, address standardisation using a reference address database, and product catalogue normalisation to ensure that the same physical product could not exist under multiple SKU codes across systems. The result was the elimination of five of the six data reconciliation roles within three months of go-live, with the remaining staff member redeployed as a data stewardship analyst.

Key Results

DATA OPS HEADCOUNT

−83%

6 STAFF REDUCED TO 1 DATA STEWARD

PROJECT ROI

3.5 mo

FULL RETURN ON INVESTMENT

ORDER ERROR RATE

1.2%

DOWN FROM 12% (90% REDUCTION)

CUSTOMER 360 VIEW

Real-time

DOWN FROM 48-HOUR COMPILATION

Client Testimonial

> *We eliminated an entire department's worth of manual data entry. The integration hub ABC Company built pays for itself every single month in recovered staff time alone, and the reduction in order errors has materially improved our relationship with two of our largest hospital accounts who were on the verge of switching to a competitor after one too many invoicing disputes.*
>
> **— COO, GlobalMed Supplies**

About ABC Company

ABC Company is a software engineering consultancy headquartered in Colombo, Sri Lanka, with delivery hubs in London and Melbourne. We partner with startups, scale-ups, and enterprise organisations to design and build digital products and platforms across a wide range of industries and technology stacks.

Our engagements are structured around a core belief: that the best software outcomes come from teams with genuine ownership, deep technical craft, and a clear-eyed understanding of the business problem they are solving. We do not staff projects with generalists reading from a methodology playbook. We build focused, experienced teams around each client's specific technical and commercial context.

What We Do Best

From greenfield platform development and legacy modernisation, through regulatory compliance implementations and enterprise system integration, to real-time data architectures and machine learning-powered product features — the eight case studies in this document represent the breadth and depth of what ABC Company delivers.

Engagement Model

We offer three primary engagement models, selected based on the client's needs and the nature of the work:

Dedicated Team	A fully managed, dedicated engineering team embedded in your product development process. Best for product companies building their core platform.
Project-Based	A scoped, milestone-driven engagement with a fixed or time-and-materials commercial structure. Best for well-defined projects with clear deliverables.
Technical Advisory	Senior architectural and technical leadership embedded in your existing team. Best for organisations needing to level up their engineering capability or navigate a complex technical decision.

Contact

To discuss how ABC Company can support your next engineering initiative, please contact our business development team:

Email	hello@abccompany.io
Website	www.abccompany.io
Colombo	Level 12, World Trade Centre, Colombo 01, Sri Lanka
London	2nd Floor, 1 Fore Street Avenue, London EC2Y 9DT, UK
Melbourne	Level 10, 530 Collins Street, Melbourne VIC 3000, Australia